ГУАП

КАФЕДРА № [ВПИСАТЬ КАФЕДРУ]

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ № 2

Модульная архитектура и контракт программного модуля

по МДК.01.04 «Системное программирование»

РАБОТУ ВЫПОЛНИЛ
Студент гр. № [ВПИСАТЬ ГРУППУ]

	подпись, дата	Д. Карачебан

Санкт-Петербург

2026

ВВЕДЕНИЕ

Цель работы — выделить в собственном участке HitPipe модуль с ясной ответственностью, поведенческим контрактом и заменяемыми зависимостями. Предмет исследования — проверка существующей прикладной сессии IAM.

Задачи: определить допустимые входы и ошибки до реализации, подключить модуль к прикладному потребителю, показать две реализации одной зависимости, проверить направления импортов и обосновать локальное архитектурное решение.

1 СВЕДЕНИЯ О РАБОТЕ И ПРОЕКТЕ

Поле	**Значение**
Автор	Дмитрий Карачебан
Проект	HitPipe — сопровождение бизнес-операций между цифровыми сервисами
Индивидуальный участок	IAM: проверка прикладной сессии; каталог services/iam
Роль в команде	Руководитель проекта; аутентификация, авторизация и профиль
Рецензент	Тимофей Патрушев
Период работы	18–19 октября 2026 года
Репозиторий	hitpipe/hitpipe
Интегрированная версия	main, commit 73aef8d976c9f0676ee568d8dc5a82c8d35a2f00; PR №12–19 объединены 19.10.2026

2 ЦЕЛЬ, ПРОЕКТ И МЕСТО МОДУЛЯ

Требования HP-BP-001/T/002–003 и DK-FR-01–03 связывают потребность оператора в безопасном доступе с конкретной операцией Evaluate(snapshot, observation). Прикладной сценарий AuthenticateSession.Service.Execute получает сведения через SessionLookup, затем считывает Clock и вызывает предметную проверку. Диагностическая команда собирает эти зависимости и отображает результат.

Модуль не выдаёт и не обновляет сессии, не выполняет OIDC-вход и не проверяет разрешение бизнес-действия. Согласованность SQL-хранилища, актуальность членства в рабочей области и барьер отзыва доступа относятся к другим обязанностям IAM. Такое разделение позволяет проверить правило времени и состояний независимо от транспорта и хранения.

3 ХОД РАБОТЫ

3.1 Контракт и документация до реализации

Контракт уточнён в PR №13, commit 9d7c37151606656e9dc2f2f65876b8bd93b397cd, до реализации в PR №14. Канонический локальный документ — services/iam/docs/module-contract.md. Комментарии GoDoc описывают назначение публичных типов, владение значениями и категории ошибок.

Часть контракта	**Принятое поведение**
Вход предметной функции	Snapshot: Found, Revoked, CreatedAt, ExpiresAt, IdleExpiresAt; Observation: At и Trusted. Передаются значения, а не изменяемое хранилище.
Ссылка на входные сведения	От 1 до 128 ASCII-символов: первый — буква или цифра, следующие — буквы, цифры, подчёркивание или дефис. Это внутренний диагностический reference, а не формат cookie или сетевого Session UUID.
Корректность времени	Ненулевые значения с представимым UTC-годом 1–9999; created<idle<=absolute; created<absolute; absolute-created<=24 часа; now>=created.
Истечение срока	now>=deadline означает истечение. Льготного интервала нет. Текущий idle-срок принадлежит источнику сессии; без lastActivity его нельзя заново ограничить created+8 часов.
Приоритет причин	Недоверенное время → неизвестная сессия → отзыв → неверная хронология → создание в будущем → абсолютный срок → срок бездействия → действующая сессия.

Часть контракта	**Принятое поведение**
Прикладные зависимости	SessionLookup.Lookup(ctx, ref) и Clock.Now(ctx). Интерфейсы определяет потребитель; адаптеры реализуют эти интерфейсы.
Ошибки	Отсутствующие и типизированные nil-зависимости отвергаются при создании; нулевой Service недоступен. Неверный reference/nil context дают отказ, отмена и сбой Lookup/Clock — недоступность.
Свежесть	Clock читается после успешного Lookup. Актуальность реального snapshot обязан обеспечивать адаптер источника.
Владение	Входы и результаты — значения; изменяемые указатели не передаются наружу. Service не хранит состояние запросов. Конкурентная безопасность зависимостей — обязательство их владельца.
Побочные эффекты	Domain не читает файлы, среду или часы и не изменяет вход. Прикладной сценарий не обновляет сессию. Файловый адаптер только читает синтетический JSON.
Безопасный результат	Ограниченный набор state/reason. Не выводятся исходные ошибки адаптера или парсера, reference, профиль и секреты. Valid не означает разрешение бизнес-действия.

Пример: сессия создана в 00:00 UTC, idle-срок — 08:00, абсолютный — следующие 00:00. В 07:59:59.999999999 при доверенном времени результат valid; ровно в 08:00 — denied/idle_expired. Если истекли оба срока, выбирается absolute_expired. Ошибка Lookup даёт unavailable/lookup_unavailable, ошибка Clock — unavailable/clock_unavailable.

Публичный контракт и существенные правила SessionPolicy

> Версия исходников: 73aef8d976c9f0676ee568d8dc5a82c8d35a2f00

Реальная реализация policy.go

```
70: func Evaluate(snapshot Snapshot, observation Observation)
Decision {
71:     if !observation.Usable() {
72:             return deny(ReasonClockUncertain)
73:     }
74:     if !snapshot.Found {
75:             return deny(ReasonUnknown)
76:     }
77:     if snapshot.Revoked {
78:             return deny(ReasonRevoked)
79:     }
80:     created := snapshot.CreatedAt.UTC()
81:     absolute := snapshot.ExpiresAt.UTC()
82:     idle := snapshot.IdleExpiresAt.UTC()
83:     now := observation.At.UTC()
84:     if !usableTime(created) || !usableTime(absolute) ||
```

Рисунок 1 - Публичный контракт и существенная реализация (часть 1 из 2)

```
!usableTime(idle) ||
85:             !absolute.After(created) || !idle.After(created)
|| idle.After(absolute) ||
86:             absolute.Sub(created) > MaxAbsoluteLifetime {
87:             return deny(ReasonInvalidSnapshot)
88:     }
89:     if now.Before(created) {
90:             return deny(ReasonNotYetValid)
91:     }
92:     if !now.Before(absolute) {
93:             return deny(ReasonAbsoluteExpired)
94:     }
95:     if !now.Before(idle) {
96:             return deny(ReasonIdleExpired)
97:     }
98:     return Decision{State: StateValid, Reason: ReasonValid}
99: }
```

Браузерный просмотр фактических файлов. Журнал показан выдержкой; полный оригинал и его SHA-256 сохранены в пакете. Полный контракт: services/iam/docs/module-contract.md. Потребитель Service.Execute вызывает этот Evaluate; это не отдельная учебная копия.

Рисунок 1 - Публичный контракт и существенная реализация (часть 2 из 2)

Документация модуля: go doc

> Версия исходников: 73aef8d976c9f0676ee568d8dc5a82c8d35a2f00
> Время UTC: 2026-10-19T17:39:40.256190+00:00
> $ go -C services/iam doc ./internal/domain/sessionpolicy
> Фактический exit: 0
> $ go -C services/iam doc ./internal/application/authenticatesession
> Фактический exit: 0

SessionPolicy

```
package sessionpolicy // import
"github.com/hitpipe/hitpipe/services/iam/internal/domain/sessionp
olicy"

Package sessionpolicy evaluates a minimal authority-owned session
snapshot.
It performs no IO, reads no clock and grants no business
authorization.

const MaxAbsoluteLifetime = 24 * time.Hour
type Decision struct{ ... }
    func Evaluate(snapshot Snapshot, observation Observation)
Decision
type Observation struct{ ... }
```

Рисунок 2 - Документация модуля по стандарту Go (часть 1 из 2)

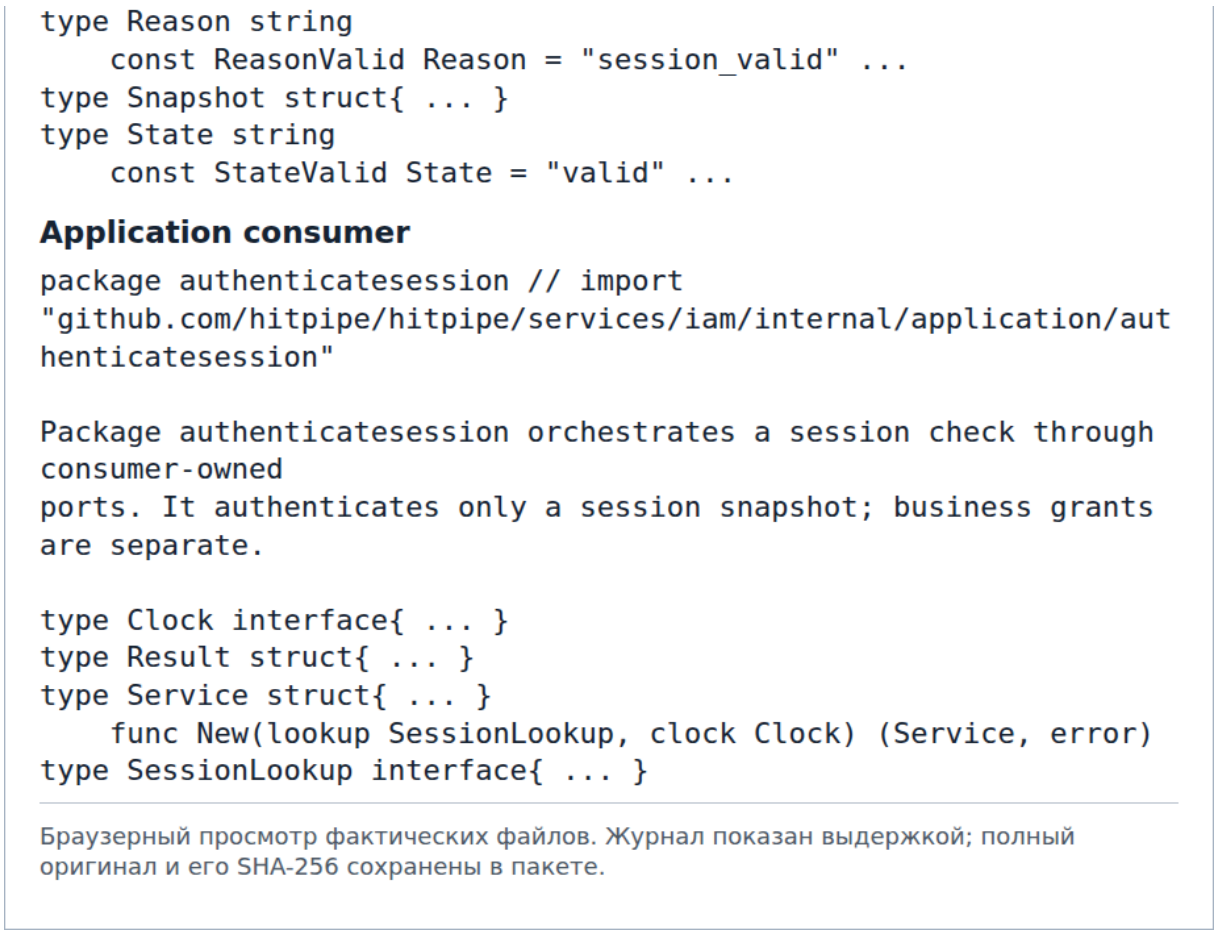


Рисунок 2 - Документация модуля по стандарту Go (часть 2 из 2)

3.2 Реализация и интеграция

Внутренняя структура следует подходу ports and adapters. Предметная функция находится в internal/domain/sessionpolicy, прикладной сценарий и его интерфейсы — в internal/application/authenticatesession. Файловый и часовой адаптеры расположены отдельно, а cmd/iam-contractcheck является точкой сборки.

PR №14 добавил чистое правило и диагностику, PR №15 — прикладного потребителя и адаптеры, PR №16 — расширенные проверки. Предметное правило не скопировано в CLI: реальная команда вызывает Service.Execute, который использует ту же Evaluate.

В опубликованном техническом ревью Тимофея Патрушева код PR №12 –18 получил положительную оценку в пределах рассмотренного участка. Два замечания к отчётному пакету — каталог запуска Go-команд и перенос строк

таблиц — исправлены. Изменения объединены в основную ветку; обязательные проверки IAM, Intake, Processing и Delivery завершились успешно.

3.3 Две реализации за одной границей

Service.Execute зависит от SessionLookup. NewFile при обращении читает и строго разбирает синтетический JSON; NewFixed возвращает неизменяемую копию переданного значения. Переключение выполняется в точке сборки и не требует изменения потребителя.

Обе реализации предназначены для ограниченного диагностического сценария с одним snapshot. Они не являются производственным многосессионным хранилищем. FixedClock возвращает заданное наблюдение; изменение файла само по себе не делает это время актуальным. Для SystemClock требуется явный HealthProbe; отсутствующая или отрицательная проверка исправности приводит к недоступности, а не к фиктивному успеху.

Один consumer, две подключаемые реализации

> Версия исходников: 73aef8d976c9f0676ee568d8dc5a82c8d35a2f00
> Время UTC: 2026-10-19T17:39:40.248783+00:00
> $ .pr12-build/iam/release/iam-contractcheck --fixture services/iam/testdata/valid.json --format json
> Фактический exit: 0
> $ .pr12-build/iam/release/iam-contractcheck --lookup fixed --fixture services/iam/testdata/valid.json --format json
> Фактический exit: 0

Настоящая точка сборки

```
65:     var lookup authenticatesession.SessionLookup
66:     if *lookupMode == "fixed" {
67:             lookup = sessionlookup.NewFixed(snapshot, false)
68:     } else {
69:             lookup, err = sessionlookup.NewFile(*path)
70:             if err != nil {
71:                     return fail()
72:             }
73:     }
74:     var clock authenticatesession.Clock =
sessionlookup.NewFixedClock(observation)
75:     if *clockMode == "system" {
```

Рисунок 3 - Один потребитель и две реализации зависимости (часть 1 из 2)

```
76:             clock = systemclock.New(nil)
77:     }
78:     service, err := authenticatesession.New(lookup, clock)
79:     if err != nil {
80:             return fail()
81:     }
82:     decision := service.Execute(context.Background(),
"diagnostic")
83:     if *format == "json" {
```

File Lookup — фактический результат

```
{"state":"valid","reason":"session_valid"}
iam-contractcheck event=start
iam-contractcheck event=end status=valid
```

Fixed Lookup — фактический результат

```
{"state":"valid","reason":"session_valid"}
iam-contractcheck event=start
iam-contractcheck event=end status=valid
```

Браузерный просмотр фактических файлов. Журнал показан выдержкой; полный оригинал и его SHA-256 сохранены в пакете. Оба запуска используют Service.Execute. Адаптеры синтетические; текущая бизнес-авторизация и live authority этим не подтверждаются.

Рисунок 3 - Один потребитель и две реализации зависимости (часть 2 из 2)

3.4 Карта модулей

Путь внутри services/iam	**internal/domain/sessionpolicy**
Ответственность	Evaluate, Snapshot, Observation, Decision; проверка переданных значений
Прямые внутренние зависимости	Нет; используется стандартный пакет time

Путь внутри services/iam	**internal/application/authenticatesession**
Ответственность	New, Service.Execute, SessionLookup, Clock, Result; порядок обращения к зависимостям

Путь внутри services/iam	**internal/application/authenticatesession**
Прямые внутренние зависимости	domain/sessionpolicy

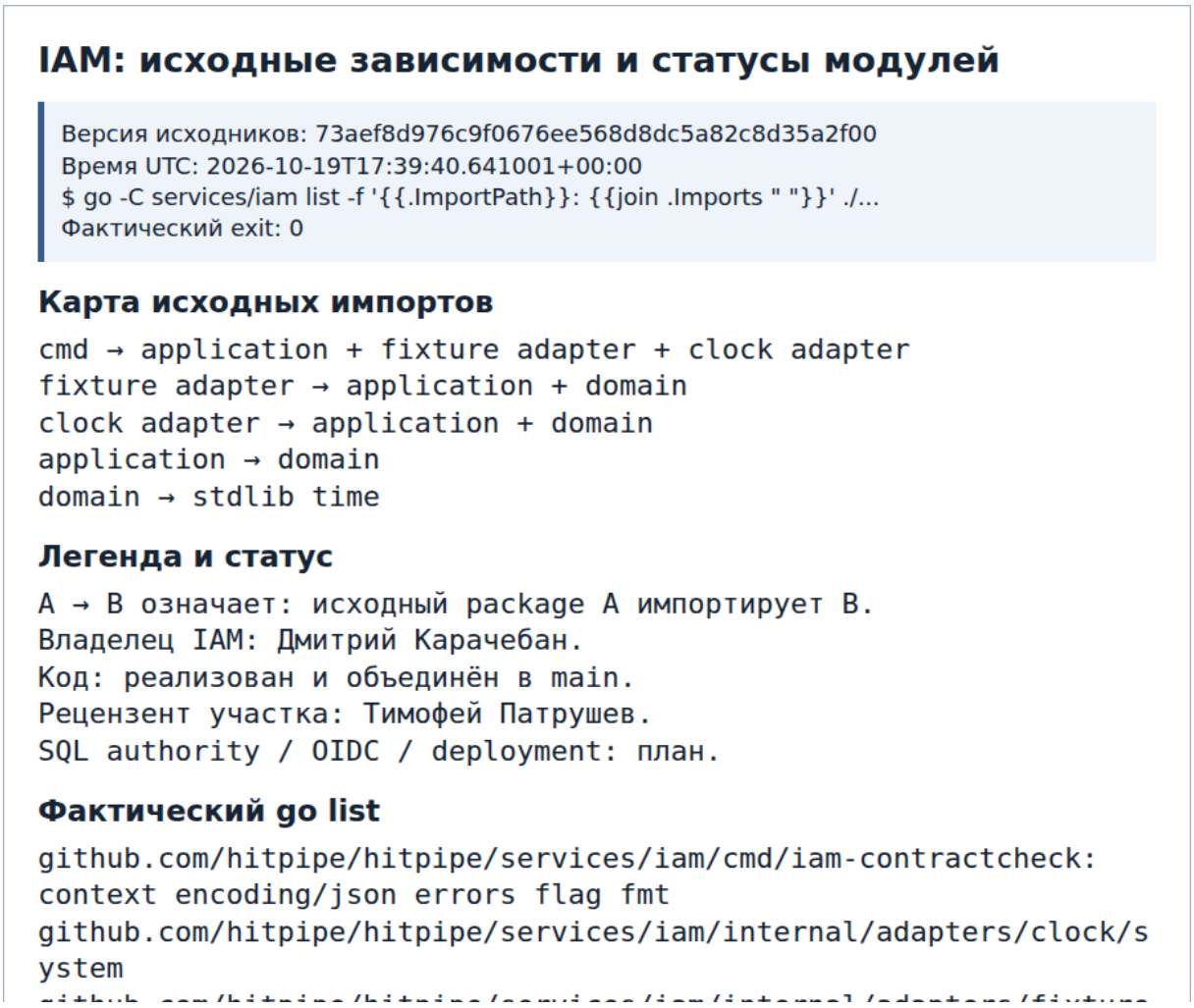
Путь внутри services/iam	**internal/adapters/fixtures/sessionlookup**
Ответственность	Decode, NewFile, NewFixed, NewFixedClock; синтетический ввод и подмена
Прямые внутренние зависимости	application/authenticatesession, domain/sessionpolicy

Путь внутри services/iam	**internal/adapters/clock/system**
Ответственность	Чтение системных часов через явно переданный HealthProbe
Прямые внутренние зависимости	application/authenticatesession, domain/sessionpolicy

Путь внутри services/iam	**cmd/iam-contractcheck**
Ответственность	Выбор реализаций, сборка зависимостей, вызов Execute, безопасный вывод и завершение
Прямые внутренние зависимости	Прикладной пакет и оба адаптера

Стрелка A→B означает, что исходный пакет A импортирует B. Направление исходных зависимостей отличается от направления вызовов интерфейса во время исполнения: прикладной код вызывает порт, но не

импортирует конкретный адаптер. Проверка go list и сборка не обнаружили циклов или импортов внутренностей другого сервиса. Владелец перечисленных модулей — Дмитрий Карачебан.



IAM: исходные зависимости и статусы модулей

> Версия исходников: 73aef8d976c9f0676ee568d8dc5a82c8d35a2f00
> Время UTC: 2026-10-19T17:39:40.641001+00:00
> $ go -C services/iam list -f '{{.ImportPath}}: {{join .Imports " "}}' ./...
> Фактический exit: 0

Карта исходных импортов

```
cmd → application + fixture adapter + clock adapter
fixture adapter → application + domain
clock adapter → application + domain
application → domain
domain → stdlib time
```

Легенда и статус

```
A → B означает: исходный package A импортирует B.
Владелец IAM: Дмитрий Карачебан.
Код: реализован и объединён в main.
Рецензент участка: Тимофей Патрушев.
SQL authority / OIDC / deployment: план.
```

Фактический go list

```
github.com/hitpipe/hitpipe/services/iam/cmd/iam-contractcheck:
context encoding/json errors flag fmt
github.com/hitpipe/hitpipe/services/iam/internal/adapters/clock/s
ystem
```

Рисунок 4 - Модульная карта с импортами, автором и статусами (часть 1 из 2)

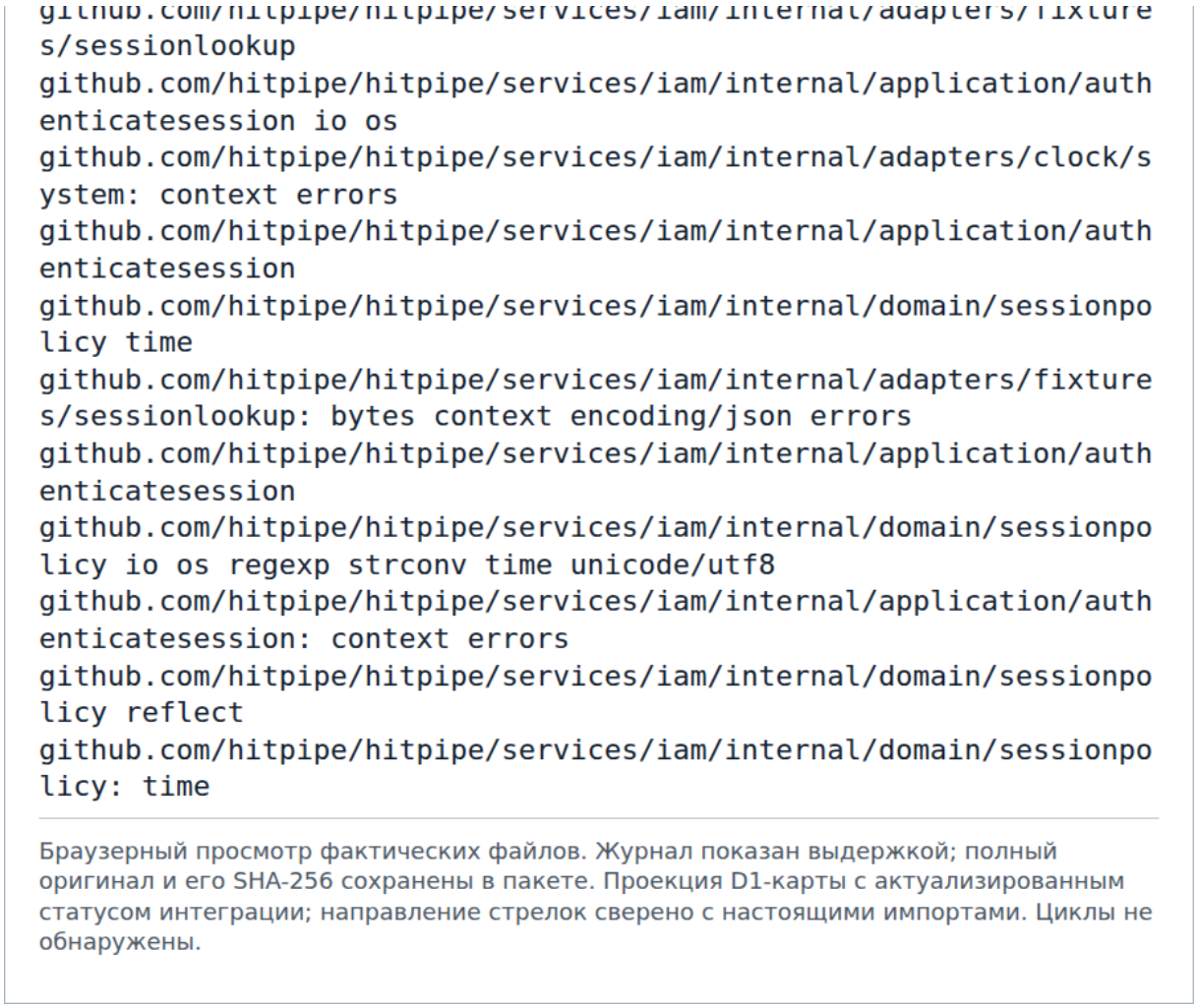


Рисунок 4 - Модульная карта с импортами, автором и статусами (часть 2 из 2)

3.5 Профиль сложности и собственное решение

Первое направление сложности — взаимодействие компонентов. После ожидания Lookup нельзя использовать устаревшее время или превращать ошибку в сохранённый успешный ответ. Существующий кандидат исследования — Service.Execute с портами Lookup/Clock. Тесты задержанного чтения, ошибки и отмены проверяют требуемый порядок. Следующий шаг — реальный источник с договорённостями о таймауте и свежести данных.

Второе направление — представление и обработка состояния. Минимальный snapshot должен сохранять владение значениями, точную границу истечения и однозначный порядок причин. Исследуемый объект — Evaluate с фиксированными датами и таблицей решений. Граничные,

конкурентные и проверяющие неизменность тесты выполнены. SQL-согласованность и долговечность относятся к будущему адаптеру хранения.

Третье, перспективное направление — стоимость копирования временных значений и разбора JSON. Для него существуют исполняемый кандидат и воспроизводимые входы, но узкое место не установлено. Измерения benchmark, CPU/alloc pprof и trace имеют смысл после определения представительной нагрузки; размер бинарника не заменяет этих измерений.

Собственное решение DK-ADR-001, автор Дмитрий Карачебан, сравнивает три варианта: правило непосредственно в CLI и прямой time.Now; немедленное подключение SQL/OIDC; чистое предметное правило с портами и адаптерами. Выбран третий вариант. Он обеспечивает повторное использование и детерминированную проверку, а ценой являются дополнительные интерфейсы, проверка nil-зависимостей и необходимость настоящих адаптеров на следующем этапе. Решение подтверждается картой импортов, подменой, тестами ошибок и границ, детектором гонок и мутационными проверками.

4 РЕЗУЛЬТАТЫ

4.1 Поведенческая проверка обеих реализаций

TestSameConsumerContractWithBothAdapters запускает один Service.Execute с File и Fixed. Для каждой реализации исполнены normal, idle_equal, unknown, revoked и error. Итого проверены не менее трёх требуемых категорий — штатная, ошибочная и граничная — для двух реализаций.

Категория	**Штатная**
Ожидание для File и Fixed	valid/session_valid
Результат	Успешно для обеих реализаций

Категория	**Равенство idle-срока**
Ожидание для File и Fixed	denied/idle_expired
Результат	Успешно для обеих реализаций

Категория	**Неизвестная сессия**
Ожидание для File и Fixed	denied с причиной неизвестности
Результат	Успешно для обеих реализаций

Категория	**Отозванная сессия**
Ожидание для File и Fixed	denied с причиной отзыва
Результат	Успешно для обеих реализаций

Категория	**Ошибка чтения**
Ожидание для File и Fixed	unavailable/lookup_unavailable
Результат	Успешно для обеих реализаций

Дополнительно проверены повторное чтение File, независимость Fixed от последующего изменения исходного значения, 32 параллельных чтения, типизированный nil, ошибка часов и отмена контекста. Локальные проверки и CI прошли; детектор гонок не обнаружил конфликтов в исполненных сценариях.



Один consumer: normal, error и boundary для обоих адаптеров

Версия исходников: 73aef8d976c9f0676ee568d8dc5a82c8d35a2f00
Время UTC: 2026-10-19T17:39:40.306603+00:00
$ go -C services/iam test ./internal/adapters/fixtures/sessionlookup -run TestSameConsumerContractWithBothAdapters -v -count=1
Фактический exit: 0

Фактический вывод теста

```
--- PASS: TestSameConsumerContractWithBothAdapters (0.00s)
    --- PASS:
TestSameConsumerContractWithBothAdapters/file/normal (0.00s)
    --- PASS:
TestSameConsumerContractWithBothAdapters/file/idle_equal (0.00s)
    --- PASS:
TestSameConsumerContractWithBothAdapters/file/unknown (0.00s)
    --- PASS:
TestSameConsumerContractWithBothAdapters/file/revoked (0.00s)
    --- PASS: TestSameConsumerContractWithBothAdapters/file/error
(0.00s)
    --- PASS:
TestSameConsumerContractWithBothAdapters/fixed/normal (0.00s)
    --- PASS:
TestSameConsumerContractWithBothAdapters/fixed/idle_equal (0.00s)
    --- PASS:
TestSameConsumerContractWithBothAdapters/fixed/unknown (0.00s)
    --- PASS:
TestSameConsumerContractWithBothAdapters/fixed/revoked (0.00s)
    --- PASS:
TestSameConsumerContractWithBothAdapters/fixed/error (0.00s)
PASS
ok
github.com/hitpipe/hitpipe/services/iam/internal/adapters/fixture
s/sessionlookup 0.005s
```

Браузерный просмотр фактических файлов. Журнал показан выдержкой; полный оригинал и его SHA-256 сохранены в пакете.

Рисунок 5 - Штатный, ошибочный и граничный случай обеих реализаций

4.2 Техническая проверка модуля другого участника

19 октября 2026 года на интегрированной версии 73aef8d976c9f0676ee568d8dc5a82c8d35a2f00 выполнено техническое воспроизведение компонента Processing, за который отвечает Тимофей

Патрушев. Использовано отдельное чистое рабочее дерево репозитория; проверка не требует ключей внешних сервисов или персональных данных.

Из корня репозитория выполнены проверка Processing с детектором гонок, диагностический запуск в режимах dev/release, тест TestReaderSubstitution и получение списка импортов. Все четыре команды завершились с кодом 0. Для File-реализации и тестовой подмены SourceFactsReader проверены штатный, ошибочный и граничные случаи. Карта зависимостей сопоставлена с фактическими импортами; циклов не обнаружено. Полные команды, параметры среды, время и результаты сохранены в evidence/peer-technical.

Рассмотрены контракт PaidOrderEligibility, потребитель EvaluatePaidOrder и подмена SourceFactsReader. Результаты относятся к объявленному локальному сценарию Processing. Они не подтверждают живое взаимодействие с платёжным провайдером.

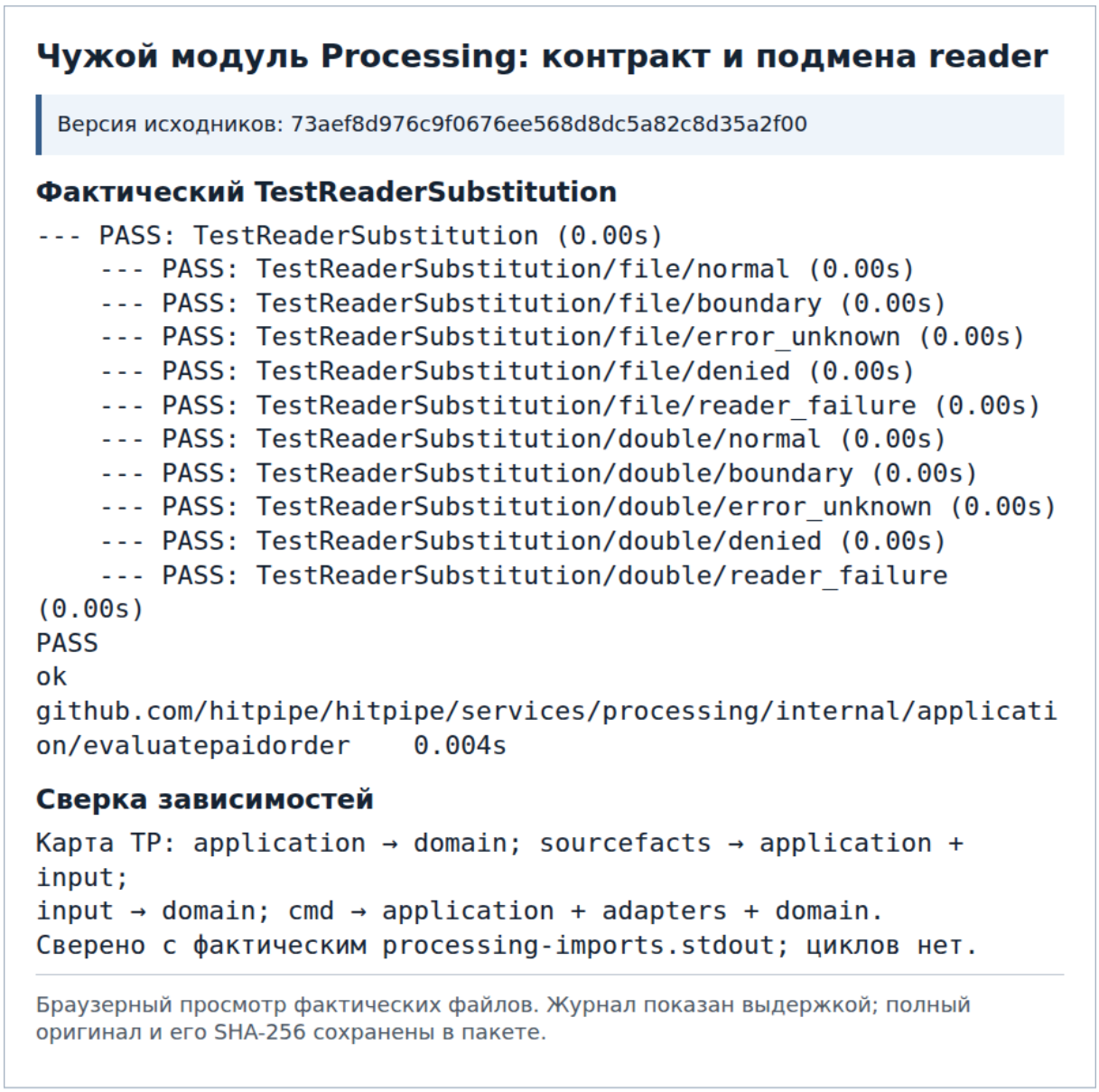


Рисунок 6 - Проверка контракта и подмены в Processing

5 ЗАКЛЮЧЕНИЕ И ИНДИВИДУАЛЬНЫЙ ВКЛАД

Модуль Дмитрия Карачебана имеет определённую ответственность, документированный контракт, существенную реализацию и реального потребителя. Конкретные зависимости заменяются в точке сборки; потребитель остаётся неизменным. Подтверждены штатные, ошибочные и граничные исходы двух реализаций.

Карта импортов соответствует исходникам, владение данными и побочные эффекты определены. Профиль сложности и DK-ADR-001 задают проверяемое основание для дальнейшего развития IAM. Настоящие адаптеры хранения и аутентификации могут быть добавлены за установленными границами без переноса предметного правила в инфраструктурный слой.

СПИСОК ИСПОЛЬЗОВАННЫХ МАТЕРИАЛОВ

1. Методические указания к практическим работам №1 «Подготовка инструментального контура программного проекта» и №2 «Модульная архитектура» по МДК01.04. Актуальные редакции, предоставленные преподавателем.

2. Методические указания к практическим работам №1 «Требования и регламент» и №2 «Проектирование и реализация системы проверок программного продукта» по МДК03.02. Актуальные редакции, предоставленные преподавателем.

3. Проект HitPipe: описание MVP, регламент изменений, решение о технологическом стеке LAB-STACK-001; архитектурные материалы HP-BP-001, разделы T/002, T/003, T/013, и HP-BP-007, раздел T/005. Копии исходных документов и контрольные суммы — docs/labs/pr12/upstream и source-files.json.

4. Исходный код и документация IAM: module-contract.md, policy.go, service.go, адаптеры, диагностическая команда и тесты.

5. Материалы индивидуального участка Дмитрия Карачебана: требования, карта модулей, DK-ADR-001, проектирование тестов, результаты и реестр рисков.

6. Изменение требований, PR №13, реализация модуля, PR №14, итоги технического ревью, PR №19.

7. Успешный запуск CI для проверенной версии IAM. Журналы повторных запусков на интегрированной версии и исходные изображения включены в электронные материалы работы.